Assignment: Fleet Vehicle Maintenance Management API

Technology Stack

Use the following technologies:

- ASP.NET Core Web API 8.0
- Entity Framework Core
- SQL Server
- Repository Pattern
- Service Pattern
- DTOs
- Pagination
- Sorting
- Filtering
- Swagger for API testing

Business Scenario

A logistics company owns many delivery vehicles. Each vehicle periodically goes for service and maintenance. The company wants a Web API to manage:

- Vehicles
- Drivers
- Maintenance records

The admin should be able to view maintenance records with pagination, sorting, and filtering because the table may contain thousands of records.

Database Name

```
FleetMaintenanceDb
```

Tables Required

1. Vehicles

```
CREATE TABLE Vehicles
(
    VehicleId INT IDENTITY(1,1) PRIMARY KEY,
    VehicleNumber VARCHAR(20) NOT NULL,
    VehicleType VARCHAR(50) NOT NULL,
    Brand VARCHAR(50) NOT NULL,
    Model VARCHAR(50) NOT NULL,
    PurchaseYear INT NOT NULL,
    IsActive BIT NOT NULL
);
```

2. Drivers

```
CREATE TABLE Drivers
(
    DriverId INT IDENTITY(1,1) PRIMARY KEY,
    DriverName VARCHAR(100) NOT NULL,
    LicenseNumber VARCHAR(50) NOT NULL,
    PhoneNumber VARCHAR(15) NOT NULL,
    City VARCHAR(50) NOT NULL,
    IsAvailable BIT NOT NULL
);
```

3. MaintenanceRecords

```
CREATE TABLE MaintenanceRecords
(
    MaintenanceId INT IDENTITY(1,1) PRIMARY KEY,
    VehicleId INT NOT NULL,
    DriverId INT NOT NULL,
    ServiceDate DATE NOT NULL,
    ServiceType VARCHAR(100) NOT NULL,
    ServiceCost DECIMAL(18,2) NOT NULL,
    ServiceStatus VARCHAR(30) NOT NULL,
    Remarks VARCHAR(250) NULL,
    CreatedDate DATETIME NOT NULL,

    CONSTRAINT FK_MaintenanceRecords_Vehicles
```

```
    FOREIGN KEY (VehicleId) REFERENCES Vehicles(VehicleId),

    CONSTRAINT FK_MaintenanceRecords_Drivers
    FOREIGN KEY (DriverId) REFERENCES Drivers(DriverId)
);
```

Sample Data Requirement

Insert minimum:

- 10 vehicles
- 10 drivers
- 30 maintenance records

The data should be meaningful and valid.

Example service statuses:

```
Scheduled
InProgress
Completed
Cancelled
```

Example service types:

```
Oil Change
Brake Inspection
Engine Repair
Tyre Replacement
Battery Check
General Service
```

Project Structure

Create the ASP.NET Core Web API project using the following structure:

```
FleetMaintenanceApi
│
├── Controllers
```

```
│   ├── VehiclesController.cs
│   ├── DriversController.cs
│   └── MaintenanceRecordsController.cs
│
├── Data
│   └── FleetMaintenanceDbContext.cs
│
├── Models
│   ├── Vehicle.cs
│   ├── Driver.cs
│   └── MaintenanceRecord.cs
│
├── DTOs
│   ├── VehicleCreateDto.cs
│   ├── VehicleResponseDto.cs
│   ├── DriverCreateDto.cs
│   ├── DriverResponseDto.cs
│   ├── MaintenanceCreateDto.cs
│   ├── MaintenanceResponseDto.cs
│   ├── MaintenanceFilterRequestDto.cs
│   └── PagedResponseDto.cs
│
├── Repositories
│   ├── Interfaces
│   │   ├── IVehicleRepository.cs
│   │   ├── IDriverRepository.cs
│   │   └── IMaintenanceRepository.cs
│   │
│   └── Implementations
│       ├── VehicleRepository.cs
│       ├── DriverRepository.cs
│       └── MaintenanceRepository.cs
│
├── Services
│   ├── Interfaces
│   │   ├── IVehicleService.cs
│   │   ├── IDriverService.cs
│   │   └── IMaintenanceService.cs
│   │
│   └── Implementations
│       ├── VehicleService.cs
│       ├── DriverService.cs
│       └── MaintenanceService.cs
│
├── Program.cs
└── appsettings.json
```

Functional Requirements

Vehicle Endpoints

Create the following endpoints:

1. Get All Vehicles

```
GET /api/vehicles
```

Expected response:

```
{
  "statusCode": 200,
  "message": "Vehicles retrieved successfully",
  "data": []
}
```

2. Get Vehicle by Id

```
GET /api/vehicles/{id}
```

Validation:

- If id is less than or equal to 0, return BadRequest.
- If vehicle is not found, return NotFound.

3. Add Vehicle

```
POST /api/vehicles
```

Sample request:

```
{
  "vehicleNumber": "TN38AB1234",
  "vehicleType": "Truck",
  "brand": "Tata",
  "model": "Ace",
  "purchaseYear": 2021,
```

```
  "isActive": true
}
```

Validations:

- Vehicle number is required.
- Vehicle type is required.
- Brand is required.
- Purchase year must be greater than 2000.

Driver Endpoints

4. Get All Drivers

```
GET /api/drivers
```

5. Get Driver by Id

```
GET /api/drivers/{id}
```

6. Add Driver

```
POST /api/drivers
```

Sample request:

```
{
  "driverName": "Ramesh Kumar",
  "licenseNumber": "DL2026TN1001",
  "phoneNumber": "9876543210",
  "city": "Coimbatore",
  "isAvailable": true
}
```

Validations:

- Driver name is required.
- License number is required.
- Phone number is required.

- City is required.

Maintenance Endpoints

7. Add Maintenance Record

```
POST /api/maintenanceRecords
```

Sample request:

```
{
  "vehicleId": 1,
  "driverId": 2,
  "serviceDate": "2026-06-15",
  "serviceType": "Oil Change",
  "serviceCost": 2500,
  "serviceStatus": "Scheduled",
  "remarks": "Regular oil replacement"
}
```

Validations:

- VehicleId must exist.
- DriverId must exist.
- Service date is required.
- Service cost must be greater than 0.
- Service status is required.

Main Pagination, Filtering, and Sorting Endpoint

8. Get Maintenance Records with Pagination, Filtering, and Sorting

```
GET /api/maintenanceRecords
```

This endpoint should support query string parameters.

Query Parameters

```
pageNumber
pageSize
vehicleId
driverId
serviceStatus
fromDate
toDate
sortBy
sortDirection
```

Example API URLs

Get First Page

```
GET /api/maintenanceRecords?pageNumber=1&pageSize=5
```

Filter by Vehicle

```
GET /api/maintenanceRecords?vehicleId=1
```

Filter by Driver

```
GET /api/maintenanceRecords?driverId=2
```

Filter by Service Status

```
GET /api/maintenanceRecords?serviceStatus=Completed
```

Filter by Date Range

```
GET /api/maintenanceRecords?fromDate=2026-06-01&toDate=2026-06-30
```

Sort by Service Date

```
GET /api/maintenanceRecords?sortBy=serviceDate&sortDirection=asc
```

Sort by Service Cost

```
GET /api/maintenanceRecords?sortBy=serviceCost&sortDirection=desc
```

Sort by Vehicle Number

```
GET /api/maintenanceRecords?sortBy=vehicleNumber&sortDirection=asc
```

Sort by Driver Name

```
GET /api/maintenanceRecords?sortBy=driverName&sortDirection=asc
```

Full Example

```
GET /api/maintenanceRecords?
pageNumber=1&pageSize=5&serviceStatus=Completed&sortBy=serviceDate&sortDirection=desc
```

Allowed Sort Fields

The API should allow sorting only by these fields:

```
maintenanceId
serviceDate
serviceType
serviceCost
serviceStatus
vehicleNumber
driverName
createdDate
```

If the user sends an invalid sort field, return BadRequest.

Example:

```
GET /api/maintenanceRecords?sortBy=salary
```

Expected response:

```
{
  "statusCode": 400,
  "message": "Invalid sort field"
}
```

Allowed Sort Directions

The API should allow only:

```
asc
desc
```

If the user sends an invalid sort direction, return BadRequest.

Example:

```
GET /api/maintenanceRecords?sortBy=serviceDate&sortDirection=up
```

Expected response:

```
{
  "statusCode": 400,
  "message": "Invalid sort direction. Allowed values are asc and desc"
}
```

Pagination Rules

Validations

- PageNumber must be greater than 0.

- PageSize must be greater than 0.
- PageSize should not be greater than 100.

Invalid example:

```
GET /api/maintenanceRecords?pageNumber=0&pageSize=10
```

Expected response:

```
{
  "statusCode": 400,
  "message": "Page number must be greater than zero"
}
```

Invalid example:

```
GET /api/maintenanceRecords?pageNumber=1&pageSize=500
```

Expected response:

```
{
  "statusCode": 400,
  "message": "Page size cannot be greater than 100"
}
```

DTO Requirements

MaintenanceFilterRequestDto

```
public class MaintenanceFilterRequestDto
{
    public int PageNumber { get; set; } = 1;

    public int PageSize { get; set; } = 10;

    public int? VehicleId { get; set; }

    public int? DriverId { get; set; }
```

```
    public string? ServiceStatus { get; set; }

    public DateOnly? FromDate { get; set; }

    public DateOnly? ToDate { get; set; }

    public string? SortBy { get; set; } = "serviceDate";

    public string? SortDirection { get; set; } = "asc";
}
```

PagedResponseDto

```
public class PagedResponseDto<T>
{
    public int StatusCode { get; set; }

    public string Message { get; set; } = string.Empty;

    public int PageNumber { get; set; }

    public int PageSize { get; set; }

    public int TotalRecords { get; set; }

    public int TotalPages { get; set; }

    public bool HasPreviousPage { get; set; }

    public bool HasNextPage { get; set; }

    public List<T> Data { get; set; } = new List<T>();
}
```

Repository Pattern Requirements

Create repository interfaces and implementations.

IVehicleRepository

Methods required:

```
Task<List<Vehicle>> GetAllVehiclesAsync();

Task<Vehicle?> GetVehicleByIdAsync(int vehicleId);

Task AddVehicleAsync(Vehicle vehicle);

Task<bool> VehicleExistsAsync(int vehicleId);
```

IDriverRepository

Methods required:

```
Task<List<Driver>> GetAllDriversAsync();

Task<Driver?> GetDriverByIdAsync(int driverId);

Task AddDriverAsync(Driver driver);

Task<bool> DriverExistsAsync(int driverId);
```

IMaintenanceRepository

Methods required:

```
IQueryable<MaintenanceRecord> GetMaintenanceRecordsQueryable();

Task AddMaintenanceRecordAsync(MaintenanceRecord maintenanceRecord);
```

Service Pattern Requirements

Create service interfaces and implementations.

IVehicleService

Methods required:

```
Task<List<VehicleResponseDto>> GetAllVehiclesAsync();

Task<VehicleResponseDto?> GetVehicleByIdAsync(int vehicleId);
```

```
Task<(bool Success, string Message, VehicleResponseDto? Data)>
AddVehicleAsync(VehicleCreateDto vehicleCreateDto);
```

IDriverService

Methods required:

```
Task<List<DriverResponseDto>> GetAllDriversAsync();

Task<DriverResponseDto?> GetDriverByIdAsync(int driverId);

Task<(bool Success, string Message, DriverResponseDto? Data)>
AddDriverAsync(DriverCreateDto driverCreateDto);
```

IMaintenanceService

Methods required:

```
Task<(bool Success, string Message, PagedResponseDto<MaintenanceResponseDto>?
Data)>
    GetPagedMaintenanceRecordsAsync(MaintenanceFilterRequestDto filter);

Task<(bool Success, string Message, MaintenanceResponseDto? Data)>
    AddMaintenanceRecordAsync(MaintenanceCreateDto maintenanceCreateDto);
```

Maintenance Repository Query Requirement

The maintenance repository should return `IQueryable` because filtering, sorting, count, and pagination should be applied before executing the database query.

Example:

```
public IQueryable<MaintenanceRecord> GetMaintenanceRecordsQueryable()
{
    return _context.MaintenanceRecords
        .Include(m => m.Vehicle)
        .Include(m => m.Driver)
```

```
        .AsQueryable();
}
```

Pagination Logic Requirement

Use:

```
int totalRecords = await query.CountAsync();

int totalPages = (int)Math.Ceiling(totalRecords / (double)filter.PageSize);

List<MaintenanceRecord> records = await query
    .Skip((filter.PageNumber - 1) * filter.PageSize)
    .Take(filter.PageSize)
    .ToListAsync();
```

Sorting Logic Requirement

The service should support sorting using switch expression.

Example:

```
query = filter.SortBy?.ToLower() switch
{
    "maintenanceid" => isDescending
        ? query.OrderByDescending(m => m.MaintenanceId)
        : query.OrderBy(m => m.MaintenanceId),

    "servicedate" => isDescending
        ? query.OrderByDescending(m => m.ServiceDate)
        : query.OrderBy(m => m.ServiceDate),

    "servicecost" => isDescending
        ? query.OrderByDescending(m => m.ServiceCost)
        : query.OrderBy(m => m.ServiceCost),

    "vehiclenumber" => isDescending
        ? query.OrderByDescending(m => m.Vehicle.VehicleNumber)
        : query.OrderBy(m => m.Vehicle.VehicleNumber),
```

```
    "drivername" => isDescending
        ? query.OrderByDescending(m => m.Driver.DriverName)
        : query.OrderBy(m => m.Driver.DriverName),

    _ => query.OrderBy(m => m.ServiceDate)
};
```

Filtering Logic Requirement

The API should support:

Filter by Vehicle

```
if (filter.VehicleId.HasValue)
{
    query = query.Where(m => m.VehicleId == filter.VehicleId.Value);
}
```

Filter by Driver

```
if (filter.DriverId.HasValue)
{
    query = query.Where(m => m.DriverId == filter.DriverId.Value);
}
```

Filter by Service Status

```
if (!string.IsNullOrWhiteSpace(filter.ServiceStatus))
{
    query = query.Where(m =>
        m.ServiceStatus.ToLower() == filter.ServiceStatus.ToLower());
}
```

Filter by Date Range

```
if (filter.FromDate.HasValue)
{
    query = query.Where(m => m.ServiceDate >= filter.FromDate.Value);
}
```

```
if (filter.ToDate.HasValue)
{
    query = query.Where(m => m.ServiceDate <= filter.ToDate.Value);
}
```

Expected Maintenance Response


```
{
  "statusCode": 200,
  "message": "Maintenance records retrieved successfully",
  "pageNumber": 1,
  "pageSize": 5,
  "totalRecords": 30,
  "totalPages": 6,
  "hasPreviousPage": false,
  "hasNextPage": true,
  "data": [
    {
      "maintenanceId": 1,
      "vehicleId": 1,
      "vehicleNumber": "TN38AB1234",
      "vehicleType": "Truck",
      "driverId": 2,
      "driverName": "Ramesh Kumar",
      "serviceDate": "2026-06-15",
      "serviceType": "Oil Change",
      "serviceCost": 2500,
      "serviceStatus": "Completed",
      "remarks": "Regular oil replacement",
      "createdDate": "2026-06-01T10:30:00"
    }
  ]
}
```

Controller Requirements

Create three controllers:

VehiclesController

Required endpoints:

```
GET /api/vehicles
GET /api/vehicles/{id}
POST /api/vehicles
```

DriversController

Required endpoints:

```
GET /api/drivers
GET /api/drivers/{id}
POST /api/drivers
```

MaintenanceRecordsController

Required endpoints:

```
GET /api/maintenanceRecords
POST /api/maintenanceRecords
```

Program.cs Requirements

Register:

```
builder.Services.AddScoped<IVehicleRepository, VehicleRepository>();
builder.Services.AddScoped<IDriverRepository, DriverRepository>();
builder.Services.AddScoped<IMaintenanceRepository, MaintenanceRepository>();

builder.Services.AddScoped<IVehicleService, VehicleService>();
builder.Services.AddScoped<IDriverService, DriverService>();
builder.Services.AddScoped<IMaintenanceService, MaintenanceService>();
```

Also configure:

```
builder.Services.AddDbContext<FleetMaintenanceDbContext>(options =>
{

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
});
```

Testing Requirements

Test the following using Swagger:

Vehicle Testing

```
GET /api/vehicles
GET /api/vehicles/1
POST /api/vehicles
```

Driver Testing

```
GET /api/drivers
GET /api/drivers/1
POST /api/drivers
```

Maintenance Testing

```
GET /api/maintenanceRecords?pageNumber=1&pageSize=5
GET /api/maintenanceRecords?serviceStatus=Completed
GET /api/maintenanceRecords?vehicleId=1
GET /api/maintenanceRecords?driverId=2
GET /api/maintenanceRecords?fromDate=2026-06-01&toDate=2026-06-30
GET /api/maintenanceRecords?sortBy=serviceCost&sortDirection=desc
GET /api/maintenanceRecords?
pageNumber=1&pageSize=5&serviceStatus=Completed&sortBy=serviceDate&sortDirection=desc
POST /api/maintenanceRecords
```

Validation Test Cases

Invalid Page Number

```
GET /api/maintenanceRecords?pageNumber=0&pageSize=10
```

Expected:

```
{
  "statusCode": 400,
  "message": "Page number must be greater than zero"
}
```

Invalid Page Size

```
GET /api/maintenanceRecords?pageNumber=1&pageSize=0
```

Expected:

```
{
  "statusCode": 400,
  "message": "Page size must be greater than zero"
}
```

Invalid Sort Field

```
GET /api/maintenanceRecords?sortBy=salary
```

Expected:

```
{
  "statusCode": 400,
  "message": "Invalid sort field"
}
```

Invalid Sort Direction

```
GET /api/maintenanceRecords?sortBy=serviceDate&sortDirection=up
```

Expected:

```
{
  "statusCode": 400,
  "message": "Invalid sort direction. Allowed values are asc and desc"
}
```

Candidate Tasks

The candidate should complete the following:

1. Create database and tables in SQL Server.
2. Insert valid sample data.
3. Create ASP.NET Core Web API 8.0 project.
4. Scaffold or create EF Core models.
5. Create DTO classes.
6. Create repository interfaces.
7. Create repository implementations.
8. Create service interfaces.
9. Create service implementations.
10. Create controllers.
11. Register dependencies in `Program.cs`.
12. Implement filtering.
13. Implement sorting.
14. Implement pagination.
15. Test all endpoints in Swagger.
16. Validate all failure scenarios.

Expected Learning Outcome

After completing this assignment, the candidate should be able to:

- Build ASP.NET Core Web API using layered architecture.
- Implement Repository Pattern.
- Implement Service Pattern.
- Use DTOs instead of exposing entity models directly.

- Use `IQueryable` for dynamic query building.
- Implement database-level filtering.
- Implement database-level sorting.
- Implement database-level pagination.
- Return paged API responses with metadata.
- Validate query string parameters.
- Test REST APIs using Swagger.

Bonus Tasks

Bonus 1: Multiple Field Sorting

Support:

```
GET /api/maintenanceRecords?sortBy=serviceDate,serviceCost&sortDirection=desc
```

Bonus 2: Search by Vehicle Number

Support:

```
GET /api/maintenanceRecords?vehicleNumber=TN38
```

Bonus 3: Search by Driver Name

Support:

```
GET /api/maintenanceRecords?driverName=Ramesh
```

Bonus 4: Update Maintenance Status

Create endpoint:

```
PUT /api/maintenanceRecords/{id}/status
```

Sample request:

```
{
  "serviceStatus": "Completed"
}
```

Bonus 5: Delete Maintenance Record

Create endpoint:

```
DELETE /api/maintenanceRecords/{id}
```

Final Submission

Submit:

- SQL script file
- ASP.NET Core Web API source code
- Swagger screenshots
- API test URLs
- Sample request and response screenshots
- Short explanation of Repository Pattern and Service Pattern used in the project